

University of North Carolina at Charlotte

User Manual

ITSC 3155 - Software Engineering

Team Alpha

Date: 7/30/2026

GitHub: https://github.com/mmallare/ITSC3155_GroupProject

Overview:

The API we created powers the backend for a sandwich ordering and management system. It handles the application from start to finish, starting with menu and ingredient management, to customer accounts, order placement/tracking, payments, promotions, and business analytics.

Built within PyCharm/VSCode, being centered in a GitHub environment, using a RESTful interface built with Fast API and a backend MySQL database.

The primary features include:

Menu and Recipe Management - defining sandwiches and linking them to ingredients (resources/recipes)

Inventory Tracking - resources/resource_management track ingredient stock

Order Processing/Tracking - orders/order_details give the sandwich name and quantity. The tracking_number and status field give unique codes to track orders and a status on the order

Customer Accounts - customers store the contact and account info of a customer, linking with customer reviews, payments, and item cart.

Reviews/Ratings - allows customers to leave a review tied to their account

Payments - payment_information stores payment methods and statuses linked to the customers ID

Shopping Cart - the cart.py links a customer to menu items with the quantity and subtotal with the option for coupon/promotions

Promotion/Discount - the promotions support discount codes for the customer to use

Business Statistics - this tracks menu item metrics. This includes order count, ratings, money spent, and peak traffic times

Setup-Up Instructions:

In order to properly setup and start running the API, you must first clone the GitHub repository into your preferred IDE. Once the program is pulled into an IDE, you must download the dependencies as follows:

```
pip install fastapi
```

```
pip install "uvicorn[standard]"
```

```
pip install sqlalchemy
```

```
pip install pymysql
```

```
pip install pytest
```

```
pip install pytest-mock
```

```
pip install httpx
```

```
pip install cryptography
```

After running those commands in the terminal, you can proceed to run the server:

```
uvicorn api.main:app --reload
```

Once the server is running, you can check it out via built-in docs on the browser:

<http://127.0.0.1:8000/docs>

To test the API endpoints and tables, here's the recommended chronological order of steps to take to ensure all tables talk to each other properly. We will split them into tiers in order to efficiently test and input data.

Tier 1:

POST /sandwiches/ -> create a few sandwiches

POST /resources/ -> create a few resources for each sandwich

POST /resource-management/ -> not dependent on other tables

POST /customers/ -> create a few customers for testing

POST /menu-items/ -> Create a few menu items for the products

POST /promotions/ -> not dependent on other tables, but used to apply discounts to total price

POST /orders/ -> creating an order creates a tracking number and order date for separate orders

Tier 2:

POST /recipes/ -> requires sandwich id and resource id from previous tier

POST /orderdetails/ -> requires order id and sandwich id from previous tier

POST /ratings-reviews/ -> requires customer id from previous tier

POST /payment-information/ -> requires customer id from previous tier

POST /statistics/ -> requires menu item id from previous tier

Tier 3:

POST /carts/ -> requires customer id and menu item id from Tier 1

Read/Update/Delete Operations:

Each table declared in our project has a read, update, and delete function. The read operation allows us to read all or read a specific value for each table. The Update function allows us to update existing information stored in the table. The delete operation allows us to delete information. The read and delete operations are tied to the unique ID listed for each option. To update, it is also tied to the information's unique ID, however, we can still alter the input body in any way we want.

Usage Examples (Example Pictures as follows: Write example: Green background -> Read Example: Blue Background):

For each API endpoint, you get the option to GET (read a value, can choose between read all or read one based on certain ID criteria), POST (write a value), PUT (update the information), or DELETE (remove information entirely). There is a certain exception for the Orders endpoint,

you can read an item based on the ID or by the tracking number that is generated. You are also given the option to update the status of a customer's order. Below are examples of what you can expect when you want to write information, including the writing description and the expected response body. The response body shown is the read value, the description shown is where you input information when creating information. A note to keep in mind, some of these tables are dependent on each other as listed in the set-up instructions.

Orders:

```
{
  "customer_name": "string",
  "description": "string"
}
```

```
{
  "customer_name": "Test Name",
  "description": "extra pickles",
  "id": 2,
  "order_date": "2026-07-12T22:06:40",
  "tracking_number": "de75a207-8313-438e-9281-1b73688058c6",
  "status": "received",
  "order_details": [
    {
      "amount": 3,
      "id": 2,
      "order_id": 2,
      "sandwich": {
        "sandwich_name": "Chicken",
        "price": 6,
        "id": 2
      }
    }
  ]
}
```

Order Details:

Edit Value | Schema

```
{
  "amount": 0,
  "order_id": 0,
  "sandwich_id": 0
}
```

Example Value | Schema

```
[
  {
    "amount": 0,
    "id": 0,
    "order_id": 0,
    "sandwich": {
      "sandwich_name": "string",
      "price": 0,
      "id": 0
    }
  }
]
```

Carts:

[Edit Value](#) | [Schema](#)

```
{
  "subtotal": 0,
  "coupon": "string",
  "quantity": 0,
  "customer_id": 0,
  "menu_item_id": 0
}
```

```
[{"subtotal": "000000000000000000000000000000000000000000872299260554861912395889917960412010  
81946209524542886521594557149199177421148171816365159269369423409799007890426175042758221186278828079066845748521474613  
35884858",  
coupon: "string",  
quantity: 0,  
customer_id: 0,  
menu_item_id: 0,  
id: 0  
}]
```

Payment Information:

Example Value | Schema

```
{
  "card_info": "string",
  "status": "string",
  "payment_type": "string",
  "customer_id": 0
}
```

Example Value | Schema

```
[
  {
    "card_info": "string",
    "status": "string",
    "payment_type": "string",
    "customer_id": 0,
    "id": 0
  }
]
```

Recipes:

Example Value	Schema
<pre>{ "amount": 0, "id": 0, "sandwich": { "sandwich_name": "string", "price": 0, "id": 0 }, "resource": { "item": "string", "amount": 0, "id": 0 } }</pre>	

Example Value	Schema
<pre>[{ "amount": 0, "id": 0, "sandwich": { "sandwich_name": "string", "price": 0, "id": 0 }, "resource": { "item": "string", "amount": 0, "id": 0 } }]</pre>	

Resources:

Example Value	Schema
<pre>{ "item": "string", "amount": 0 }</pre>	

Example Value	Schema
<pre>[{ "item": "string", "amount": 0, "id": 0 }]</pre>	

Sandwiches:

Example Value	Schema
<pre>{ "sandwich_name": "string", "price": 0 }</pre>	

Example Value	Schema
<pre>[{ "sandwich_name": "string", "price": 0, "id": 0 }]</pre>	

Statistics:

Example Value	Schema
---------------	--------

```
{
  "menu_item_id": 0,
  "menu_order_count": 0,
  "rating_score": 0,
  "avg_money_spent": 0,
  "peak_hours_traffic": "string",
  "frequency": 0
}
```

Example Value	Schema
---------------	--------

[illegible]

Menu Items:

Example Value	Schema
1	int
1.1	float
1.1e10	float
1.1e-10	float
1.1e+10	float
1.1e-10e10	float
1.1e-10e-10	float
1.1e-10e+10	float
1.1e+10e10	float
1.1e+10e-10	float
1.1e+10e+10	float
1.1e-10e10e10	float
1.1e-10e-10e-10	float
1.1e-10e+10e10	float
1.1e+10e10e10	float
1.1e+10e-10e-10	float
1.1e+10e+10e+10	float
1.1e-10e10e-10	float
1.1e-10e-10e10	float
1.1e-10e+10e-10	float
1.1e-10e-10e+10	float
1.1e-10e+10e+10	float
1.1e+10e10e-10	float
1.1e+10e-10e10	float
1.1e+10e+10e10	float
1.1e-10e10e+10	float
1.1e-10e+10e10	float
1.1e+10e10e+10	float
1.1e-10e10e-10e10	float
1.1e-10e-10e10e10	float
1.1e-10e+10e10e10	float
1.1e+10e10e10e10	float
1.1e+10e-10e-10e10	float
1.1e+10e+10e-10e10	float
1.1e+10e10e-10e-10	float
1.1e+10e-10e-10e-10	float
1.1e+10e+10e-10e+10	float
1.1e+10e-10e+10e+10	float
1.1e+10e+10e-10e+10	float
1.1e+10e+10e+10e+10	float
1.1e-10e10e-10e-10	float
1.1e-10e-10e10e-10	float
1.1e-10e+10e10e-10	float
1.1e-10e-10e10e+10	float
1.1e-10e+10e10e+10	float
1.1e+10e10e-10e+10	float
1.1e+10e-10e+10e+10	float
1.1e+10e+10e-10e+10	float
1.1e+10e+10e+10e+10	float
1.1e-10e10e-10e+10	float
1.1e-10e-10e10e+10	float
1.1e-10e+10e10e+10	float
1.1e+10e10e-10e+10	float
1.1e+10e-10e+10e+10	float
1.1e+10e+10e-10e+10	float
1.1e+10e+10e+10e+10	float
1.1e-10e10e-10e+10e10	float
1.1e-10e-10e10e+10e10	float
1.1e-10e+10e10e+10e10	float
1.1e+10e10e-10e+10e10	float
1.1e+10e-10e+10e+10e10	float
1.1e+10e+10e-10e+10e10	float
1.1e+10e+10e+10e+10e10	float
1.1e-10e10e-10e+10e10e10	float
1.1e-10e-10e10e+10e10e10	float
1.1e-10e+10e10e+10e10e10	float
1.1e+10e10e-10e+10e10e10	float
1.1e+10e-10e+10e+10e10e10	float
1.1e+10e+10e-10e+10e10e10	float
1.1e+10e+10e+10e+10e10e10	float
1.1e-10e10e-10e+10e10e10e10	float
1.1e-10e-10e10e+10e10e10e10	float
1.1e-10e+10e10e+10e10e10e10	float
1.1e+10e10e-10e+10e10e10e10	float
1.1e+10e-10e+10e+10e10e10e10	float
1.1e+10e+10e-10e+10e10e10e10	float
1.1e+10e+10e+10e+10e10e10e10	float
1.1e-10e10e-10e+10e10e10e10e10	float
1.1e-10e-10e10e+10e10e10e10e10	float
1.1e-10e+10e10e+10e10e10e10e10	float
1.1e+10e10e-10e+10e10e10e10e10	float
1.1e+10e-10e+10e+10e10e10e10e10	float
1.1e+10e+10e-10e+10e10e10e10e10	float
1.1e+10e+10e+10e+10e10e10e10e10	float
1.1e-10e10e-10e+10e10e10e10e10e10	float
1.1e-10e-10e10e+10e10e10e10e10e10	float
1.1e-10e+10e10e+10e10e10e10e10e10	float
1.1e+10e10e-10e+10e10e10e10e10e10	float
1.1e+10e-10e+10e+10e10e10e10e10e10	float
1.1e+10e+10e-10e+10e10e10e10e10e10	float
1.1e+10e+10e+10e+10e10e10e10e10e10	float
1.1e-10e10e-10e+10e10e10e10e10e10e10	float
1.1e-10e-10e10e+10e10e10e10e10e10e10	float
1.1e-10e+10e10e+10e10e10e10e10e10e10	float
1.1e+10e10e-10e+10e10e10e10e10e10e10	float
1.1e+10e-10e+10e+10e10e10e10e10e10e10	float
1.1e+10e+10e-10e+10e10e10e10e10e10e10	float
1.1e+10e+10e+10e+10e10e10e10e10e10e10	float
1.1e-10e10e-10e+10e10e10e10e10e10e10e1	

```
{
  "item_name": "string",
  "item_price": 0,
  "calories": 0
}
```

Example Value	Schema
---------------	--------

[illegible]

Resource Management:

Customers:

Example Value | Schema

```
{
  "name": "string",
  "email": "string",
  "phone": "string",
  "address": "string"
}
```

Example Value | Schema

```
[
  {
    "name": "string",
    "email": "string",
    "phone": "string",
    "address": "string",
    "id": 0
  }
]
```